```
            this._showKeyboardHints = !isTouch;
            this.refresh();
        }

        private createIconNode(
            type: WeaponType,
            def: WeaponDef,
            level: number,
            isActive: boolean,
            slotIndex: number
        ): Node {
            const node = new Node(`WeaponIcon_${type}`);
            node.layer = UI_LAYER;
            const iconSize = this._iconSize;
            node.addComponent(UITransform).setContentSize(iconSize, iconSize);

            const themeColor = this.hexToColor(def.iconColor);

            // Background
            const g = node.addComponent(Graphics);
            const radius = Math.max(10, Math.round(iconSize * 0.18));

            g.fillColor = isActive ? new Color(34, 48, 60, 252) : new Color(20, 30, 42, 232);
            g.roundRect(-iconSize / 2, -iconSize / 2, iconSize, iconSize, radius);
            g.fill();

            g.fillColor = isActive ? new Color(64, 154, 206, 58) : new Color(32, 84, 112, 42);
            g.roundRect(
                -iconSize / 2 + 4,
                -iconSize / 2 + 4,
                iconSize - 8,
                iconSize - 8,
                Math.max(8, radius - 3)
            );
            g.fill();

            g.strokeColor = isActive
                ? new Color(255, 220, 116, 255)
                : new Color(themeColor.r, themeColor.g, themeColor.b, 255);
            g.lineWidth = isActive ? 4 : 2;
            g.roundRect(-iconSize / 2, -iconSize / 2, iconSize, iconSize, radius);
            g.stroke();

            // Icon Sprite
            const iconNode = new Node('IconSprite');
            iconNode.layer = UI_LAYER;
            node.addChild(iconNode);
            const spriteSize = iconSize * 0.75;
            iconNode.addComponent(UITransform).setContentSize(spriteSize, spriteSize);
            const sprite = iconNode.addComponent(Sprite);
```

```
            sprite.sizeMode = Sprite.SizeMode.CUSTOM;

            if (def.iconPath) {
                this.loadWeaponIcon(sprite, def.iconPath);
            }

            this.createLevelBadge(node, level, iconSize, isActive);

            if (this._showKeyboardHints && slotIndex >= 0 && slotIndex < KEY_HINTS.length) {
                this.createKeyHintBadge(node, slotIndex, iconSize);
            }

            node.on(Node.EventType.TOUCH_START, () => {
                node.setScale(0.95, 0.95, 1);
            });
            node.on(Node.EventType.TOUCH_CANCEL, () => {
                node.setScale(1, 1, 1);
            });
            node.on(Node.EventType.TOUCH_END, () => {
                node.setScale(1, 1, 1);
                HeroWeaponManager.instance.switchWeapon(type);
            });

            return node;
        }

        private loadWeaponIcon(sprite: Sprite, path: string): void {
            const normalizedPath = path.trim();
            const cached = this._iconFrameCache.get(normalizedPath);
            if (cached) {
                sprite.spriteFrame = cached;
                return;
            }

            const waitingSet = this.getIconWaitingSet(normalizedPath);
            waitingSet.add(sprite);
            if (this._iconLoading.has(normalizedPath)) {
                return;
            }
            this._iconLoading.add(normalizedPath);

            const candidates = this.getIconLoadCandidates(normalizedPath);
            this.loadWeaponIconByCandidate(candidates, 0, frame => {
                this._iconLoading.delete(normalizedPath);
                const waiting = this.getIconWaitingSet(normalizedPath);
                if (!frame) {
                    console.error(`[WeaponBarUI] Failed to load weapon icon from paths:`, candidates);
                    waiting.clear();
                    return;
                }
```

```

                this._iconFrameCache.set(normalizedPath, frame);
                for (const waitingSprite of waiting) {
                    if (!waitingSprite?.isValid) continue;
                    waitingSprite.spriteFrame = frame;
                }
                waiting.clear();
            });
        }

        private getIconLoadCandidates(path: string): string[] {
            const rawCandidates = [
                path,
                `${path}/spriteFrame`,
                `${path}/texture`,
                path.endsWith('.webp') ? path : `${path}.webp`,
                path.endsWith('.webp') ? `${path}/spriteFrame` : `${path}.webp/spriteFrame`,
                path.endsWith('.webp') ? `${path}/texture` : `${path}.webp/texture`,
            ];

            const candidates: string[] = [];
            for (const candidate of rawCandidates) {
                if (!candidate || candidates.includes(candidate)) continue;
                candidates.push(candidate);
            }
            return candidates;
        }

        private loadWeaponIconByCandidate(
            candidates: string[],
            index: number,
            done: (frame: SpriteFrame | null) => void
        ): void {
            if (index >= candidates.length) {
                done(null);
                return;
            }

            const candidate = candidates[index];
            resources.load(candidate, SpriteFrame, (sfErr, spriteFrame) => {
                if (!sfErr && spriteFrame) {
                    done(spriteFrame);
                    return;
                }

                resources.load(candidate, Texture2D, (texErr, texture) => {
                    if (!texErr && texture) {
                        const frame = new SpriteFrame();
                        frame.texture = texture;
                        done(frame);
```

```
                        return;
                    }

                    resources.load(candidate, ImageAsset, (imgErr, imageAsset) => {
                        if (!imgErr && imageAsset) {
                            const textureFromImage = new Texture2D();
                            textureFromImage.image = imageAsset;
                            const frameFromImage = new SpriteFrame();
                            frameFromImage.texture = textureFromImage;
                            done(frameFromImage);
                            return;
                        }

                        this.loadWeaponIconByCandidate(candidates, index + 1, done);
                    });
                });
            });
        }

        private getIconWaitingSet(path: string): Set<Sprite> {
            let waitingSet = this._iconWaiting.get(path);
            if (!waitingSet) {
                waitingSet = new Set<Sprite>();
                this._iconWaiting.set(path, waitingSet);
            }
            return waitingSet;
        }

        private updateContainerSize(iconCount: number, isTikTokPortrait: boolean): void {
            if (!this._barNode) return;
            const columns = isTikTokPortrait ? Math.min(2, Math.max(1, iconCount)) : iconCount;
            const rows = isTikTokPortrait ? Math.max(1, Math.ceil(iconCount / 2)) : 1;
            const iconAreaWidth =
                columns > 0 ? columns * this._iconSize + Math.max(0, columns - 1) * ICON_GAP : 0;
            const iconAreaHeight =
                rows > 0 ? rows * this._iconSize + Math.max(0, rows - 1) * ICON_GAP : 0;
            const minWidth = isTikTokPortrait ? Math.round(this._iconSize * 1.4) : BAR_MIN_WIDTH;
            const minHeight = isTikTokPortrait
                ? Math.round(this._iconSize + BAR_PADDING_Y * 2 + 12)
                : BAR_MIN_HEIGHT;
            const width = Math.max(minWidth, iconAreaWidth + BAR_PADDING_X * 2);
            const height = Math.max(minHeight, iconAreaHeight + BAR_PADDING_Y * 2 + 12);
            this._barNode.getComponent(UITransform)?.setContentSize(width, height);
        }

        private createLevelBadge(
            parent: Node,
            level: number,
            iconSize: number,
            isActive: boolean
```

```
        ): void {
            const badge = new Node('LvBadge');
            badge.layer = UI_LAYER;
            parent.addChild(badge);
            badge.addComponent(UITransform).setContentSize(38, 20);
            badge.setPosition(iconSize * 0.24, -iconSize * 0.32, 0);

            const bg = badge.addComponent(Graphics);
            bg.fillColor = isActive ? new Color(255, 214, 109, 255) : new Color(52, 82, 115, 255);
            bg.roundRect(-19, -10, 38, 20, 8);
            bg.fill();
            bg.strokeColor = isActive ? new Color(255, 244, 193, 255) : new Color(126, 194, 244, 240);
            bg.lineWidth = 1.5;
            bg.roundRect(-19, -10, 38, 20, 8);
            bg.stroke();

            const labelNode = new Node('LvLabel');
            labelNode.layer = UI_LAYER;
            badge.addChild(labelNode);
            labelNode.addComponent(UITransform).setContentSize(36, 18);
            const label = labelNode.addComponent(Label);
            label.string = Localization.instance.t('ui.common.level.short', { level });
            label.fontSize = 12;
            label.lineHeight = 14;
            label.horizontalAlign = Label.HorizontalAlign.CENTER;
            label.verticalAlign = Label.VerticalAlign.CENTER;
            label.overflow = Label.Overflow.SHRINK;
            label.color = isActive ? new Color(42, 24, 10, 255) : new Color(232, 244, 255, 255);
            label.cacheMode = Label.CacheMode.NONE;
            label.useSystemFont = true;
            label.fontFamily = 'sans-serif';
        }

        private createKeyHintBadge(parent: Node, slotIndex: number, iconSize: number): void {
            const keyNode = new Node('KeyHint');
            keyNode.layer = UI_LAYER;
            parent.addChild(keyNode);
            keyNode.addComponent(UITransform).setContentSize(20, 20);
            keyNode.setPosition(-iconSize * 0.32, iconSize * 0.32, 0);

            const keyBg = keyNode.addComponent(Graphics);
            keyBg.fillColor = new Color(11, 20, 31, 255);
            keyBg.roundRect(-10, -10, 20, 20, 5);
            keyBg.fill();
            keyBg.strokeColor = new Color(255, 210, 96, 255);
            keyBg.lineWidth = 1.5;
            keyBg.roundRect(-10, -10, 20, 20, 5);
            keyBg.stroke();

            const keyLabelNode = new Node('KeyHintLabel');
```

```
            keyLabelNode.layer = UI_LAYER;
            keyNode.addChild(keyLabelNode);
            keyLabelNode.addComponent(UITransform).setContentSize(18, 18);
            const keyLabel = keyLabelNode.addComponent(Label);
            keyLabel.string = KEY_HINTS[slotIndex];
            keyLabel.fontSize = 13;
            keyLabel.lineHeight = 16;
            keyLabel.color = new Color(255, 233, 156, 255);
            keyLabel.horizontalAlign = Label.HorizontalAlign.CENTER;
            keyLabel.verticalAlign = Label.VerticalAlign.CENTER;
            keyLabel.cacheMode = Label.CacheMode.NONE;
            keyLabel.useSystemFont = true;
            keyLabel.fontFamily = 'sans-serif';
        }

        // === 工具 ===

        private hexToColor(hex: string): Color {
            if (!hex || hex.length < 7 || !hex.startsWith('#')) {
                return new Color(168, 204, 255, 255);
            }
            const r = parseInt(hex.slice(1, 3), 16);
            const g = parseInt(hex.slice(3, 5), 16);
            const b = parseInt(hex.slice(5, 7), 16);
            return new Color(r, g, b, 255);
        }

        private get eventManager(): EventManager {
            return ServiceRegistry.get<EventManager>('EventManager') ?? EventManager.instance;
        }
    }

    // ===== FILE: assets/scripts/ui/WeaponSelectUI.ts =====
    import {
        Node,
        UITransform,
        Color,
        Widget,
        Graphics,
        Label,
        Sprite,
        resources,
        SpriteFrame,
        Texture2D,
        ImageAsset,
    } from 'cc';
    import { Singleton } from '../core/base/Singleton';
    import { EventManager } from '../core/managers/EventManager';
    import { ServiceRegistry } from '../core/managers/ServiceRegistry';
    import { GameEvents } from '../data/GameEvents';
```

```
53372     import { HeroWeaponManager } from '../gameplay/weapons/HeroWeaponManager';
53373     import { WeaponType, WeaponDef } from '../gameplay/weapons/WeaponTypes';
53374     import { Localization } from '../core/i18n/Localization';
53375     import { SelectionCardTheme, type GrantToken } from './SelectionCardTheme';
53376     import { UIResponsive } from './UIResponsive';
53377     import { TikTokAdService } from '../core/ads/TikTokAdService';
53378     import { AirdropService } from '../gameplay/airdrop/AirdropService';
53379
53380     const UI_LAYER = 33554432;
53381
53382     const CARD_WIDTH = 258;
53383     const CARD_HEIGHT = 378;
53384     const CARD_GAP = 34;
53385
53386     /**
53387      * WeaponSelectUI
53388      * 空投武器选择界面（3 选 1），暂停期间展示。
53389      * 结构与 BuffCardUI 一致，保持 UI 风格统一。
53390      */
53391     export class WeaponSelectUI extends Singleton<WeaponSelectUI>() {
53392         private _uiCanvas: Node | null = null;
53393         private _rootNode: Node | null = null;
53394         private _isShowing: boolean = false;
53395         private _offeredWeaponTypes: WeaponType[] = [];
53396
53397         // Icon loading cache
53398         private _iconFrameCache: Map<string, SpriteFrame> = new Map();
53399         private _iconLoading: Set<string> = new Set();
53400         private _iconWaiting: Map<string, Set<Sprite>> = new Map();
53401
53402         public initialize(uiCanvas: Node): void {
53403             this._uiCanvas = uiCanvas;
53404             this.eventManager.on(GameEvents.WEAPONS_OFFERED, this.onWeaponsOffered, this);
53405             console.log('[WeaponSelectUI] 初始化完成');
53406         }
53407
53408         public cleanup(): void {
53409             this.eventManager.off(GameEvents.WEAPONS_OFFERED, this.onWeaponsOffered, this);
53410             this.hideCards();
53411             this._iconFrameCache.clear();
53412             this._iconLoading.clear();
53413             this._iconWaiting.clear();
53414         }
53415
53416         // === 事件处理 ===
53417
53418         private onWeaponsOffered(data: { weapons: string[] }): void {
53419             const weaponIds = data.weapons as WeaponType[];
53420             const defs: { type: WeaponType; def: WeaponDef }[] = [];
53421             const manager = HeroWeaponManager.instance;
```

```

        for (const id of weaponIds) {
            const def = manager.getWeaponDef(id);
            if (def) defs.push({ type: id, def });
        }
        if (defs.length === 0) return;

        this.showCards(defs);
    }

    // === 展示 / 隐藏 ===

    public showCards(weapons: { type: WeaponType; def: WeaponDef }[]): void {
        if (!this._uiCanvas || this._isShowing) return;
        if (
            SelectionCardTheme.isTikTokRuntime() &&
            TikTokAdService.isSessionSlotUnlocked('weapon_draw')
        ) {
            this.grantAllWeaponsAndPlayFeedback(weapons.map(item => item.type));
            return;
        }
        this._isShowing = true;
        this._offeredWeaponTypes = weapons.map(w => w.type);
        const viewport = this.getViewportSize();
        const padding = UIResponsive.getControlPadding();
        const isPortraitTikTok = UIResponsive.isTikTokPhonePortraitProfile();

        // 创建根节点（全屏遮罩）
        this._rootNode = new Node('WeaponSelectRoot');
        this._rootNode.layer = UI_LAYER;
        this._uiCanvas.addChild(this._rootNode);

        const rootTransform = this._rootNode.addComponent(UITransform);
        rootTransform.setContentSize(viewport.width, viewport.height);

        const widget = this._rootNode.addComponent(Widget);
        widget.isAlignTop = widget.isAlignBottom = widget.isAlignLeft = widget.isAlignRight = true;
        widget.top = widget.bottom = widget.left = widget.right = 0;

        // 半透明遮罩
        const maskNode = new Node('Mask');
        maskNode.layer = UI_LAYER;
        this._rootNode.addChild(maskNode);
        const maskTransform = maskNode.addComponent(UITransform);
        maskTransform.setContentSize(viewport.width, viewport.height);
        const maskWidget = maskNode.addComponent(Widget);
        maskWidget.isAlignTop =
            maskWidget.isAlignBottom =
            maskWidget.isAlignLeft =
            maskWidget.isAlignRight =
```

```
                    true;
            maskWidget.top = maskWidget.bottom = maskWidget.left = maskWidget.right = 0;
            const maskG = maskNode.addComponent(Graphics);
            SelectionCardTheme.drawOverlayMask(maskG, viewport.width, viewport.height);

            // 标题
            this.createTitle(this._rootNode, viewport.width, viewport.height);

            // 卡牌
            const totalWidth = weapons.length * CARD_WIDTH + (weapons.length - 1) * CARD_GAP;
            const usePortraitTriangle = isPortraitTikTok && weapons.length === 3;
            const triangleRowGap = 34;
            const containerWidth = usePortraitTriangle ? CARD_WIDTH * 2 + CARD_GAP : totalWidth;
            const containerHeight = usePortraitTriangle
                ? CARD_HEIGHT * 2 + triangleRowGap
                : CARD_HEIGHT;

            // Dynamic scaling if total width exceeds screen width
            const size = this._rootNode.getComponent(UITransform)?.contentSize;
            // Create a container for scaling
            const cardContainer = new Node('CardContainer');
            cardContainer.layer = UI_LAYER;
            this._rootNode.addChild(cardContainer);
            cardContainer.addComponent(UITransform).setContentSize(containerWidth, containerHeight);

            let cardScale = 1;
            if (size) {
                const availableWidth = Math.max(
                    240,
                    size.width - padding.left - padding.right - (usePortraitTriangle ? 16 : 24)
                );
                const availableHeight = Math.max(
                    180,
                    size.height - padding.top - padding.bottom - (usePortraitTriangle ? 150 : 180)
                );
                const widthScale = availableWidth / containerWidth;
                const heightScale = availableHeight / containerHeight;
                const maxScale = usePortraitTriangle ? 1.15 : 1;
                const scale = Math.min(maxScale, widthScale, heightScale);
                cardScale = scale;
                cardContainer.setScale(scale, scale, 1);
            }
            const cardContainerY = usePortraitTriangle
                ? -Math.round(viewport.height * 0.08)
                : Math.round(-padding.bottom * 0.04);
            cardContainer.setPosition(0, cardContainerY, 0);

            const startX = -totalWidth / 2 + CARD_WIDTH / 2;
            const triangleBottomX = (CARD_WIDTH + CARD_GAP) * 0.5;
            const triangleTopY = CARD_HEIGHT * 0.5 + triangleRowGap * 0.5;
```

```
            const triangleBottomY = -(CARD_HEIGHT * 0.5 + triangleRowGap * 0.5);

            weapons.forEach((w, i) => {
                const card = this.createCardNode(w, i);
                cardContainer.addChild(card);
                if (usePortraitTriangle) {
                    if (i === 0) {
                        card.setPosition(0, triangleTopY, 0);
                    } else if (i === 1) {
                        card.setPosition(-triangleBottomX, triangleBottomY, 0);
                    } else {
                        card.setPosition(triangleBottomX, triangleBottomY, 0);
                    }
                } else {
                    card.setPosition(startX + i * (CARD_WIDTH + CARD_GAP), -20, 0);
                }
                SelectionCardTheme.playCardReveal(card, i);
            });

            // 广告按钮（仅 TikTok 环境）
            const titleHeight = Math.round(
                Math.max(
                    isPortraitTikTok ? 58 : 64,
                    Math.min(
                        isPortraitTikTok ? 96 : 90,
                        viewport.height * (isPortraitTikTok ? 0.1 : 0.11)
                    )
                )
            );
            const titleTop = Math.round(
                Math.max(
                    padding.top + 8,
                    Math.min(
                        isPortraitTikTok ? 120 : 160,
                        viewport.height * (isPortraitTikTok ? 0.1 : 0.14) + padding.top * 0.2
                    )
                )
            );
            const titleBottomY = viewport.height * 0.5 - titleTop - titleHeight;
            const cardTopY = usePortraitTriangle
                ? cardContainerY + (triangleTopY + CARD_HEIGHT * 0.5) * cardScale
                : cardContainerY + (-20 + CARD_HEIGHT * 0.5) * cardScale;
            const titleToCardGap = titleBottomY - cardTopY;
            const adBtnY = Math.round(cardTopY + (titleToCardGap > 80 ? titleToCardGap * 0.5 : 40));
            const adBtnWidth = Math.round(
                Math.max(
                    260,
                    Math.min(
                        viewport.width - padding.left - padding.right - 24,
                        (CARD_WIDTH * 2 + CARD_GAP) * cardScale
```

```
                    )
                )
            );
            SelectionCardTheme.createAdButton(
                this._rootNode!,
                Localization.instance.t('ui.ad.unlock_run_all_weapons'),
                { x: 0, y: adBtnY },
                () => this.onAdButtonTapped(),
                {
                    width: adBtnWidth,
                    height: 56,
                    fontSize: 15,
                }
            );
        }

        private hideCards(): void {
            if (this._rootNode) {
                this._rootNode.destroy();
                this._rootNode = null;
            }
            this._isShowing = false;
            this._offeredWeaponTypes = [];
            // Clear waiting sets to avoid memory leaks if sprites are destroyed
            this._iconWaiting.clear();
            this._iconLoading.clear();
        }

        // === UI 构建 ===

        private createTitle(root: Node, viewportWidth: number, viewportHeight: number): void {
            const isPortraitTikTok = UIResponsive.isTikTokPhonePortraitProfile();
            const titleNode = new Node('Title');
            titleNode.layer = UI_LAYER;
            titleNode
                .addComponent(UITransform)
                .setContentSize(
                    Math.round(
                        Math.max(
                            isPortraitTikTok ? 320 : 420,
                            Math.min(
                                isPortraitTikTok ? 860 : 880,
                                viewportWidth * (isPortraitTikTok ? 0.9 : 0.72)
                            )
                        )
                    ),
                    Math.round(
                        Math.max(
                            isPortraitTikTok ? 58 : 64,
                            Math.min(
```

```
                                isPortraitTikTok ? 96 : 90,
                                viewportHeight * (isPortraitTikTok ? 0.1 : 0.11)
                            )
                        )
                    )
                );
            root.addChild(titleNode);

            // Responsive Title using Widget
            const widget = titleNode.addComponent(Widget);
            widget.isAlignTop = true;
            widget.isAlignHorizontalCenter = true;
            const padding = UIResponsive.getControlPadding();
            widget.top = Math.round(
                Math.max(
                    padding.top + 8,
                    Math.min(
                        isPortraitTikTok ? 120 : 160,
                        viewportHeight * (isPortraitTikTok ? 0.1 : 0.14) + padding.top * 0.2
                    )
                )
            );

            const label = titleNode.addComponent(Label);
            label.string = Localization.instance.t('ui.weapon.select.title');
            label.overflow = Label.Overflow.SHRINK;
            SelectionCardTheme.applyLabelTheme(label, {
                fontSize: isPortraitTikTok ? 34 : 48,
                lineHeight: isPortraitTikTok ? 40 : 54,
                color: new Color(255, 214, 92, 255),
                bold: true,
                hAlign: Label.HorizontalAlign.CENTER,
                vAlign: Label.VerticalAlign.CENTER,
                outlineColor: new Color(52, 26, 6, 255),
                outlineWidth: isPortraitTikTok ? 4 : 5,
                shadowColor: new Color(0, 0, 0, 210),
            });
            const decoNode = new Node('TitleDeco');
            decoNode.layer = UI_LAYER;
            titleNode.addChild(decoNode);
            const deco = decoNode.addComponent(Graphics);
            deco.strokeColor = new Color(255, 219, 120, 210);
            deco.lineWidth = 2;
            deco.moveTo(-230, -20);
            deco.lineTo(-95, -20);
            deco.stroke();
            deco.moveTo(95, -20);
            deco.lineTo(230, -20);
            deco.stroke();
        }
```

```

        private getViewportSize(): { width: number; height: number } {
            return UIResponsive.getLayoutViewportSize(480, 320, 'canvas');
        }

        private createCardNode(weapon: { type: WeaponType; def: WeaponDef }, _index: number): Node {
            const { type, def } = weapon;
            const manager = HeroWeaponManager.instance;
            const existing = manager.inventory.get(type);
            const currentLevel = existing ? existing.level : 0;
            const isUpgrade = currentLevel > 0;

            const cardNode = new Node(`WeaponCard_${type}`);
            cardNode.layer = UI_LAYER;
            cardNode.addComponent(UITransform).setContentSize(CARD_WIDTH, CARD_HEIGHT);

            // 颜色
            const themeColor = this.hexToColor(def.iconColor);

            // 背景
            const bg = new Node('BG');
            bg.layer = UI_LAYER;
            bg.addComponent(UITransform).setContentSize(CARD_WIDTH, CARD_HEIGHT);
            cardNode.addChild(bg);

            const g = bg.addComponent(Graphics);
            SelectionCardTheme.drawCardBackground(g, CARD_WIDTH, CARD_HEIGHT, themeColor, 78);

            // 武器名称
            const nameNode = new Node('Name');
            nameNode.layer = UI_LAYER;
            nameNode.addComponent(UITransform).setContentSize(CARD_WIDTH - 30, 56);
            cardNode.addChild(nameNode);
            const nameLabel = nameNode.addComponent(Label);
            nameLabel.string = Localization.instance.t(def.nameKey);
            SelectionCardTheme.applyLabelTheme(nameLabel, {
                fontSize: 28,
                lineHeight: 32,
                color: Color.WHITE,
                bold: true,
                hAlign: Label.HorizontalAlign.CENTER,
                vAlign: Label.VerticalAlign.CENTER,
                outlineColor: new Color(18, 20, 34, 255),
                outlineWidth: 3,
            });
            nameLabel.overflow = Label.Overflow.SHRINK;
            nameLabel.enableWrapText = false;
            nameNode.setPosition(0, CARD_HEIGHT / 2 - 42, 0);

            // 等级标签
```

```
            const levelText = isUpgrade
                ? Localization.instance.t('ui.weapon.level.upgrade', {
                      from: currentLevel,
                      to: currentLevel + 1,
                  })
                : Localization.instance.t('ui.weapon.level.new');
            SelectionCardTheme.createBadge(
                cardNode,
                levelText,
                SelectionCardTheme.blendColor(themeColor, new Color(255, 224, 146, 255), 0.3),
                { w: 146, h: 30 },
                { x: 0, y: CARD_HEIGHT / 2 - 88 },
                isUpgrade ? new Color(255, 226, 126, 255) : new Color(158, 252, 186, 255)
            );

            // 武器描述
            const descNode = new Node('Desc');
            descNode.layer = UI_LAYER;
            descNode.addComponent(UITransform).setContentSize(CARD_WIDTH - 26, 82);
            cardNode.addChild(descNode);
            const descLabel = descNode.addComponent(Label);
            descLabel.string = Localization.instance.t(def.descriptionKey);
            SelectionCardTheme.applyLabelTheme(descLabel, {
                fontSize: 17,
                lineHeight: 23,
                color: new Color(194, 208, 232, 255),
                hAlign: Label.HorizontalAlign.CENTER,
                vAlign: Label.VerticalAlign.TOP,
                outlineColor: new Color(8, 20, 32, 255),
                outlineWidth: 2,
                shadowBlur: 1,
            });
            descLabel.overflow = Label.Overflow.SHRINK;
            descLabel.enableWrapText = true;
            descNode.setPosition(0, 34, 0);

            // 武器图标 (替代原有的 Stats)
            const iconContainer = new Node('WeaponIcon');
            iconContainer.layer = UI_LAYER;
            cardNode.addChild(iconContainer);
            const containerSize = 100;
            iconContainer.addComponent(UITransform).setContentSize(containerSize, containerSize);
            iconContainer.setPosition(0, -60, 0); // Center of the bottom area

            // Icon Sprite
            const spriteNode = new Node('Sprite');
            spriteNode.layer = UI_LAYER;
            iconContainer.addChild(spriteNode);
            const spriteSize = 80;
            spriteNode.addComponent(UITransform).setContentSize(spriteSize, spriteSize);
```

```

            const sprite = spriteNode.addComponent(Sprite);
            sprite.sizeMode = Sprite.SizeMode.CUSTOM;

            if (def.iconPath) {
                this.loadWeaponIcon(sprite, def.iconPath);
            }

            // 点击
            SelectionCardTheme.bindCardClick(cardNode, () => {
                if (!this._isShowing) return;
                this.eventManager.emit(GameEvents.WEAPON_PICKED, { weaponId: type });
                this.hideCards();
            });

            return cardNode;
        }

        private onAdButtonTapped(): void {
            if (!this._isShowing || this._offeredWeaponTypes.length === 0) return;

            TikTokAdService.showRewardedAd('weapon_draw').then(rewarded => {
                if (!rewarded) {
                    if (TikTokAdService.wasLastAdCancelled()) {
                        TikTokAdService.showToast(Localization.instance.t('ui.ad.not_rewarded'));
                    }
                    return;
                }
                TikTokAdService.unlockSessionSlot('weapon_draw');
                const offered = [...this._offeredWeaponTypes];
                this.grantAllWeapons(offered);
                this.hideCards();
                this.playWeaponGrantAnimation(offered);
            });
        }

        private grantAllWeaponsAndPlayFeedback(weaponTypes: WeaponType[]): void {
            if (weaponTypes.length <= 0) return;
            this.grantAllWeapons(weaponTypes);
            this.playWeaponGrantAnimation(weaponTypes);
        }

        private grantAllWeapons(weaponTypes: WeaponType[]): void {
            const granted = this.airdropService.claimAllPendingWeapons();
            if (granted > 0) return;
            const manager = HeroWeaponManager.instance;
            for (const weaponType of weaponTypes) {
                manager.addWeapon(weaponType);
            }
        }
```

```

        private playWeaponGrantAnimation(weaponTypes: WeaponType[]): void {
            if (!this._uiCanvas || !this._uiCanvas.isValid) return;
            const manager = HeroWeaponManager.instance;
            const tokens: GrantToken[] = weaponTypes.map(type => {
                const def = manager.getWeaponDef(type);
                const accent = def ? this.hexToColor(def.iconColor) : new Color(255, 205, 96, 255);
                return {
                    text: this.getWeaponTokenText(type),
                    color: accent,
                };
            });
            const viewport = this.getViewportSize();
            SelectionCardTheme.playGrantAnimation(this._uiCanvas, {
                message: Localization.instance.t('ui.ad.auto_grant.weapons'),
                tokens,
                targetNodeName: 'WeaponBar',
                fallbackTarget: {
                    x: -Math.round(viewport.width * 0.36),
                    y: -Math.round(viewport.height * 0.36),
                },
            });
        }

        private getWeaponTokenText(type: WeaponType): string {
            if (type === WeaponType.MACHINE_GUN) return 'MG';
            if (type === WeaponType.FLAMETHROWER) return 'FL';
            if (type === WeaponType.CANNON) return 'CN';
            return 'GW';
        }

        // === Icon Loading Logic (Copied from WeaponBarUI) ===

        private loadWeaponIcon(sprite: Sprite, path: string): void {
            const normalizedPath = path.trim();
            const cached = this._iconFrameCache.get(normalizedPath);
            if (cached) {
                sprite.spriteFrame = cached;
                return;
            }

            const waitingSet = this.getIconWaitingSet(normalizedPath);
            waitingSet.add(sprite);
            if (this._iconLoading.has(normalizedPath)) {
                return;
            }
            this._iconLoading.add(normalizedPath);

            const candidates = this.getIconLoadCandidates(normalizedPath);
            this.loadWeaponIconByCandidate(candidates, 0, frame => {
```

```
                this._iconLoading.delete(normalizedPath);
                const waiting = this.getIconWaitingSet(normalizedPath);
                if (!frame) {
                    console.error(
                        `[WeaponSelectUI] Failed to load weapon icon from paths:`,
                        candidates
                    );
                    waiting.clear();
                    return;
                }

                this._iconFrameCache.set(normalizedPath, frame);
                for (const waitingSprite of waiting) {
                    if (!waitingSprite?.isValid) continue;
                    waitingSprite.spriteFrame = frame;
                }
                waiting.clear();
            });
        }

        private getIconLoadCandidates(path: string): string[] {
            const rawCandidates = [
                path,
                `${path}/spriteFrame`,
                `${path}/texture`,
                path.endsWith('.webp') ? path : `${path}.webp`,
                path.endsWith('.webp') ? `${path}/spriteFrame` : `${path}.webp/spriteFrame`,
                path.endsWith('.webp') ? `${path}/texture` : `${path}.webp/texture`,
            ];

            const candidates: string[] = [];
            for (const candidate of rawCandidates) {
                if (!candidate || candidates.includes(candidate)) continue;
                candidates.push(candidate);
            }
            return candidates;
        }

        private loadWeaponIconByCandidate(
            candidates: string[],
            index: number,
            done: (frame: SpriteFrame | null) => void
        ): void {
            if (index >= candidates.length) {
                done(null);
                return;
            }

            const candidate = candidates[index];
            resources.load(candidate, SpriteFrame, (sfErr, spriteFrame) => {
```

```
                if (!sfErr && spriteFrame) {
                    done(spriteFrame);
                    return;
                }

                resources.load(candidate, Texture2D, (texErr, texture) => {
                    if (!texErr && texture) {
                        const frame = new SpriteFrame();
                        frame.texture = texture;
                        done(frame);
                        return;
                    }

                    resources.load(candidate, ImageAsset, (imgErr, imageAsset) => {
                        if (!imgErr && imageAsset) {
                            const textureFromImage = new Texture2D();
                            textureFromImage.image = imageAsset;
                            const frameFromImage = new SpriteFrame();
                            frameFromImage.texture = textureFromImage;
                            done(frameFromImage);
                            return;
                        }

                        this.loadWeaponIconByCandidate(candidates, index + 1, done);
                    });
                });
            });
        }

        private getIconWaitingSet(path: string): Set<Sprite> {
            let waitingSet = this._iconWaiting.get(path);
            if (!waitingSet) {
                waitingSet = new Set<Sprite>();
                this._iconWaiting.set(path, waitingSet);
            }
            return waitingSet;
        }

        // === 工具 ===

        private hexToColor(hex: string): Color {
            const r = parseInt(hex.slice(1, 3), 16);
            const g = parseInt(hex.slice(3, 5), 16);
            const b = parseInt(hex.slice(5, 7), 16);
            return new Color(r, g, b, 255);
        }

        private get eventManager(): EventManager {
            return ServiceRegistry.get<EventManager>('EventManager') ?? EventManager.instance;
        }
```

```

        private get airdropService(): AirdropService {
            return ServiceRegistry.get<AirdropService>('AirdropService') ?? AirdropService.instance;
        }
    }

    // ===== FILE: devvit/src/client/preview-entry.ts =====
    import { requestExpandedMode } from '@devvit/web/client';

    document.addEventListener('DOMContentLoaded', () => {
        const playBtn = document.getElementById('play-btn');
        const bgVideo = document.getElementById('bg-video') as HTMLVideoElement | null;
        let launchInFlight = false;

        const tryPlayPreview = (): void => {
            if (!bgVideo) return;
            bgVideo.muted = true;
            bgVideo.defaultMuted = true;
            bgVideo.playsInline = true;
            bgVideo.playbackRate = 0.5;
            const maybePromise = bgVideo.play();
            if (maybePromise && typeof maybePromise.catch === 'function') {
                maybePromise.catch(() => {
                    // Ignore autoplay rejections; user interaction path still works.
                });
            }
        };

        const launchGame = async (event?: Event): Promise<void> => {
            if (launchInFlight) return;
            launchInFlight = true;
            try {
                await requestExpandedMode((event ?? new PointerEvent('click')) as PointerEvent, 'game');
            } catch (err) {
                console.error('[preview] Failed to enter expanded mode:', err);
            } finally {
                // Only debounce repeated taps while the request is in progress.
                // Releasing this lock allows Play to work again after user closes the game view.
                launchInFlight = false;
            }
        };

        if (bgVideo) {
            bgVideo.setAttribute('muted', '');
            bgVideo.setAttribute('playsinline', '');
            bgVideo.setAttribute('webkit-playsinline', '');
            bgVideo.preload = 'auto';
            bgVideo.addEventListener('loadedmetadata', () => {
                bgVideo.playbackRate = 0.5;
            });
```

```
            bgVideo.addEventListener('loadeddata', tryPlayPreview, { once: true });
            bgVideo.addEventListener('canplay', tryPlayPreview, { once: true });
            if (bgVideo.readyState >= HTMLMediaElement.HAVE_CURRENT_DATA) {
                tryPlayPreview();
            }
            document.addEventListener('visibilitychange', () => {
                if (!document.hidden) tryPlayPreview();
            });
        }

        if (playBtn) {
            playBtn.addEventListener('click', async (event: Event) => {
                event.preventDefault();
                event.stopPropagation();
                await launchGame(event);
            });
        }
    });

    // ===== FILE: devvit/src/server/index.ts =====
    import { Hono } from 'hono';
    import { serve } from '@hono/node-server';
    import { createServer, getServerPort } from '@devvit/web/server';
    import { api } from './routes/api.js';
    import { menu } from './routes/menu.js';

    const app = new Hono();

    app.route('/api', api);
    app.route('/internal/menu', menu);

    serve({
        fetch: app.fetch,
        createServer,
        port: getServerPort(),
    });

    // ===== FILE: devvit/src/server/routes/api.ts =====
    import { Hono } from 'hono';
    import type { Context } from 'hono';
    import type { ContentfulStatusCode } from 'hono/utils/http-status';
    import { context, redis, reddit } from '@devvit/web/server';

    const RANK_KEY = 'leaderboard:scores';
    const META_KEY = 'leaderboard:meta';
    const FOLLOW_KEY = 'leaderboard:followers';
    const PLAY_COUNT_KEY = 'leaderboard:playcount';
    const SCORE_RUN_PREFIX = 'leaderboard:run:';
    const RATE_LIMIT_PREFIX = 'ratelimit:submit:';
    const LEADERBOARD_SIZE = 10;
```

```
54072     const DEFAULT_STATS_LEADERBOARD_LIMIT = 200;
54073     const DEFAULT_STATS_PLAY_COUNT_LIMIT = 200;
54074     const MAX_QUERY_LIMIT = 5000;
54075
54076     // ── Diamond system keys ──────────────────────────────────────────────────
54077     const DIAMOND_BALANCE_PREFIX = 'diamond:balance:';
54078     const DIAMOND_RUN_PREFIX = 'diamond:run:';
54079     const DIAMOND_INITIAL_GRANT = 0;
54080     const SUBSCRIBE_DIAMOND_REWARD = 500;
54081     const DIAMOND_PER_WAVE = 10;
54082     const DIAMOND_ITEM_PRICE = 100;
54083     const DIAMOND_DAILY_CAP = 5000;
54084     const DIAMOND_DAILY_PREFIX = 'diamond:daily:';
54085
54086     // ── Validation constants ─────────────────────────────────────────────────
54087     const MAX_SCORE = 999_999_999;
54088     const MAX_WAVE = 9_999;
54089     const RATE_LIMIT_WINDOW_SEC = 10;
54090     const SCORE_RUN_EXPIRATION_DAYS = 30;
54091
54092     interface LeaderboardEntry {
54093         rank: number;
54094         username: string;
54095         score: number;
54096         wave: number;
54097     }
54098
54099     interface PlayCountEntry {
54100         username: string;
54101         playCount: number;
54102     }
54103
54104     async function fetchLeaderboard(limit: number = LEADERBOARD_SIZE): Promise<LeaderboardEntry[]> {
54105         if (!Number.isFinite(limit) || limit <= 0) return [];
54106         const members = await redis.zRange(RANK_KEY, 0, limit - 1, {
54107             reverse: true,
54108             by: 'rank',
54109         });
54110         if (!members || members.length === 0) return [];
54111
54112         const allMeta = (await redis.hGetAll(META_KEY)) ?? {};
54113
54114         return members.map((entry, i) => {
54115             const { member: username, score } = entry as { member: string; score: number };
54116             let wave = 0;
54117             const metaStr = allMeta[username];
54118             if (metaStr) {
54119                 try {
54120                     const parsed = JSON.parse(metaStr) as { wave?: number };
54121                     wave = parsed.wave ?? 0;
```

```
                } catch {
                    wave = 0;
                }
            }
            return { rank: i + 1, username, score: Math.round(score), wave };
        });
    }

    async function fetchPlayCounts(
        options: { reverse?: boolean; limit?: number } = {}
    ): Promise<PlayCountEntry[]> {
        const reverse = options.reverse ?? true;
        const stop = typeof options.limit === 'number' && options.limit > 0 ? options.limit - 1 : -1;
        const rows = await redis.zRange(PLAY_COUNT_KEY, 0, stop, {
            by: 'rank',
            reverse,
        });
        return (rows ?? []).map(row => {
            const r = row as { member: string; score: number };
            return {
                username: r.member,
                playCount: Math.max(0, Math.round(r.score)),
            };
        });
    }

    async function recordPlayCount(username: string): Promise<number | null> {
        try {
            const next = await redis.zIncrBy(PLAY_COUNT_KEY, username, 1);
            return Math.max(0, Math.round(next));
        } catch (error) {
            console.warn('[stats] recordPlayCount failed:', error);
            return null;
        }
    }

    async function buildStatsSummary() {
        const [uniquePlayers, playCounts] = await Promise.all([
            redis.zCard(RANK_KEY),
            fetchPlayCounts({ reverse: false }),
        ]);

        let totalRuns = 0;
        let repeatPlayers = 0;
        let singleRunPlayers = 0;
        for (const row of playCounts) {
            totalRuns += row.playCount;
            if (row.playCount > 1) repeatPlayers += 1;
            else if (row.playCount === 1) singleRunPlayers += 1;
        }
```

```

        const trackedPlayers = playCounts.length;
        const untrackedPlayers = Math.max(0, uniquePlayers - trackedPlayers);

        return {
            uniquePlayers,
            trackedPlayers,
            untrackedPlayers,
            repeatPlayers,
            singleRunPlayers,
            totalRuns,
        };
    }

    export const api = new Hono();

    function jsonNoCache(c: Context, payload: unknown, status: number = 200) {
        c.header('Cache-Control', 'no-store, no-cache, must-revalidate');
        c.header('Pragma', 'no-cache');
        c.header('Expires', '0');
        return c.json(payload, status as ContentfulStatusCode);
    }

    function jsonError(c: Context, code: number, message: string) {
        return jsonNoCache(c, { status: 'error', code, message }, code);
    }

    function isValidInt(v: unknown): v is number {
        return typeof v === 'number' && Number.isFinite(v) && Number.isInteger(v) && v >= 0;
    }

    function parseLimitQuery(raw: string | undefined, fallback: number): number {
        if (!raw) return fallback;
        const parsed = Number.parseInt(raw, 10);
        if (!Number.isFinite(parsed) || parsed <= 0) return fallback;
        return Math.min(parsed, MAX_QUERY_LIMIT);
    }

    async function checkRateLimit(username: string): Promise<boolean> {
        const key = `${RATE_LIMIT_PREFIX}${username}`;
        try {
            const expiration = new Date(Date.now() + RATE_LIMIT_WINDOW_SEC * 1000);
            const result = await redis.set(key, '1', { nx: true, expiration });
            return result === 'OK';
        } catch {
            // Redis failure → allow the request (fail-open)
            return true;
        }
    }

```

```
54222     api.get('/init', async c => {
54223         try {
54224             const leaderboardLimit = parseLimitQuery(
54225                 c.req.query('leaderboardLimit') ?? c.req.query('limit'),
54226                 LEADERBOARD_SIZE
54227             );
54228             const [username, leaderboard] = await Promise.all([
54229                 reddit.getCurrentUsername(),
54230                 fetchLeaderboard(leaderboardLimit),
54231             ]);
54232
54233             const resolvedUsername = username ?? 'Anonymous';
54234             const isSubscribed =
54235                 resolvedUsername !== 'Anonymous'
54236                     ? (await redis.hGet(FOLLOW_KEY, resolvedUsername)) === '1'
54237                     : false;
54238
54239             const diamonds =
54240                 resolvedUsername !== 'Anonymous' ? await ensureDiamondBalance(resolvedUsername) : 0;
54241
54242             return jsonNoCache(c, {
54243                 username: resolvedUsername,
54244                 isSubscribed,
54245                 subredditName: context.subredditName ?? '',
54246                 leaderboard,
54247                 leaderboardLimit,
54248                 diamonds,
54249             });
54250         } catch (error) {
54251             console.error('[api/init] error:', error);
54252             return jsonError(c, 500, 'Internal server error');
54253         }
54254     });
54255
54256     api.post('/submit-score', async c => {
54257         try {
54258             const leaderboardLimit = parseLimitQuery(
54259                 c.req.query('leaderboardLimit') ?? c.req.query('limit'),
54260                 LEADERBOARD_SIZE
54261             );
54262             const username = await reddit.getCurrentUsername();
54263             if (!username) {
54264                 return jsonError(c, 401, 'Not logged in');
54265             }
54266
54267             let body: unknown;
54268             try {
54269                 body = await c.req.json();
54270             } catch {
54271                 return jsonError(c, 400, 'Invalid JSON body');
```

```
            }

            const payload = body as { score?: unknown; wave?: unknown; runId?: unknown };
            const score = payload.score;
            const wave = payload.wave;
            const runId = payload.runId;

            // ── Input validation ──────────────────────────────────────────
            if (!isValidInt(score) || score > MAX_SCORE) {
                return jsonError(c, 400, `Invalid score: must be integer 0–${MAX_SCORE}`);
            }
            if (!isValidInt(wave) || wave > MAX_WAVE) {
                return jsonError(c, 400, `Invalid wave: must be integer 0–${MAX_WAVE}`);
            }
            if (
                runId !== undefined &&
                (typeof runId !== 'string' || runId.length < 8 || runId.length > 128)
            ) {
                return jsonError(c, 400, 'Invalid runId');
            }

            if (typeof runId === 'string') {
                const existingRun = await redis.get(`${SCORE_RUN_PREFIX}${username}:${runId}`);
                if (existingRun) {
                    return jsonNoCache(c, JSON.parse(existingRun));
                }
            }

            // ── Rate limiting ─────────────────────────────────────────────
            const allowed = await checkRateLimit(username);
            if (!allowed) {
                c.header('Retry-After', String(RATE_LIMIT_WINDOW_SEC));
                return jsonError(c, 429, 'Too many requests. Try again later.');
            }

            const existing = await redis.zScore(RANK_KEY, username);
            const isNewBest = existing == null || score > existing;

            if (isNewBest) {
                await redis.zAdd(RANK_KEY, { score, member: username });
                await redis.hSet(META_KEY, {
                    [username]: JSON.stringify({ wave, updatedAt: Date.now() }),
                });
            }

            const playCount = await recordPlayCount(username);

            const leaderboard = await fetchLeaderboard(leaderboardLimit);
            const rankIdx = leaderboard.findIndex(e => e.username === username);

```

```
            const response = {
                rank: rankIdx >= 0 ? rankIdx + 1 : LEADERBOARD_SIZE + 1,
                score,
                isNewBest,
                playCount,
                leaderboardLimit,
                leaderboard,
            };

            if (typeof runId === 'string') {
                await redis.set(`${SCORE_RUN_PREFIX}${username}:${runId}`, JSON.stringify(response), {
                    expiration: new Date(
                        Date.now() + SCORE_RUN_EXPIRATION_DAYS * 24 * 60 * 60 * 1000
                    ),
                });
            }

            return jsonNoCache(c, response);
        } catch (error) {
            console.error('[api/submit-score] error:', error);
            return jsonError(c, 500, 'Internal server error');
        }
    });

    api.get('/leaderboard', async c => {
        try {
            const limit = parseLimitQuery(c.req.query('limit'), LEADERBOARD_SIZE);
            const leaderboard = await fetchLeaderboard(limit);
            return jsonNoCache(c, { entries: leaderboard, limit });
        } catch (error) {
            console.error('[api/leaderboard] error:', error);
            return jsonNoCache(c, { entries: [] });
        }
    });

    api.get('/stats', async c => {
        try {
            const leaderboardLimit = parseLimitQuery(
                c.req.query('leaderboardLimit') ?? c.req.query('limit'),
                DEFAULT_STATS_LEADERBOARD_LIMIT
            );
            const playCountLimit = parseLimitQuery(
                c.req.query('playCountLimit'),
                DEFAULT_STATS_PLAY_COUNT_LIMIT
            );
            const [summary, leaderboard, playCounts] = await Promise.all([
                buildStatsSummary(),
                fetchLeaderboard(leaderboardLimit),
                fetchPlayCounts({ reverse: true, limit: playCountLimit }),
            ]);
```

```

            return jsonNoCache(c, {
                metrics: summary,
                leaderboard,
                playCounts,
                limits: {
                    leaderboardLimit,
                    playCountLimit,
                    maxQueryLimit: MAX_QUERY_LIMIT,
                },
                generatedAt: new Date().toISOString(),
            });
        } catch (error) {
            console.error('[api/stats] error:', error);
            return jsonError(c, 500, 'Internal server error');
        }
    });

    // ── Diamond System ──────────────────────────────────────────────────────

    async function ensureDiamondBalance(username: string): Promise<number> {
        const key = `${DIAMOND_BALANCE_PREFIX}${username}`;
        const existing = await redis.get(key);
        if (existing != null) {
            return parseInt(existing, 10) || 0;
        }
        // First-time user: grant initial diamonds
        await redis.set(key, String(DIAMOND_INITIAL_GRANT));
        return DIAMOND_INITIAL_GRANT;
    }

    function todayKey(): string {
        const d = new Date();
        return `${d.getUTCFullYear()}${String(d.getUTCMonth() + 1).padStart(2, '0')}${String(
            d.getUTCDate()
        ).padStart(2, '0')}`;
    }

    api.get('/diamond/balance', async c => {
        try {
            const username = await reddit.getCurrentUsername();
            if (!username) return jsonError(c, 401, 'Not logged in');
            const balance = await ensureDiamondBalance(username);
            return jsonNoCache(c, { balance });
        } catch (error) {
            console.error('[api/diamond/balance] error:', error);
            return jsonError(c, 500, 'Internal server error');
        }
    });

```

```
    api.post('/diamond/settle-run', async c => {
        try {
            const username = await reddit.getCurrentUsername();
            if (!username) return jsonError(c, 401, 'Not logged in');

            let body: unknown;
            try {
                body = await c.req.json();
            } catch {
                return jsonError(c, 400, 'Invalid JSON');
            }

            const payload = body as { runId?: string; wave?: unknown };
            const runId = payload.runId;
            const wave = payload.wave;

            if (!runId || typeof runId !== 'string' || runId.length < 8 || runId.length > 64) {
                return jsonError(c, 400, 'Invalid runId');
            }
            if (!isValidInt(wave) || (wave as number) > MAX_WAVE) {
                return jsonError(c, 400, 'Invalid wave');
            }

            // Idempotency: check if this runId was already settled
            const runKey = `${DIAMOND_RUN_PREFIX}${username}:${runId}`;
            const alreadySettled = await redis.get(runKey);
            if (alreadySettled) {
                const prev = JSON.parse(alreadySettled) as { earned: number; balance: number };
                return jsonNoCache(c, { earned: prev.earned, balance: prev.balance, duplicate: true });
            }

            const earned = Math.max(0, (wave as number) * DIAMOND_PER_WAVE);

            // Daily cap check
            const dailyKey = `${DIAMOND_DAILY_PREFIX}${username}:${todayKey()}`;
            const dailyUsedStr = await redis.get(dailyKey);
            const dailyUsed = dailyUsedStr ? parseInt(dailyUsedStr, 10) || 0 : 0;
            const actualEarned = Math.min(earned, Math.max(0, DIAMOND_DAILY_CAP - dailyUsed));

            // Credit diamonds
            const balanceKey = `${DIAMOND_BALANCE_PREFIX}${username}`;
            const currentBalance = await ensureDiamondBalance(username);
            const newBalance = currentBalance + actualEarned;
            await redis.set(balanceKey, String(newBalance));

            // Update daily counter (expire after 48h for safety)
            await redis.set(dailyKey, String(dailyUsed + actualEarned), {
                expiration: new Date(Date.now() + 48 * 3600 * 1000),
            });

```

```
            // Mark run as settled (expire after 7 days)
            await redis.set(runKey, JSON.stringify({ earned: actualEarned, balance: newBalance }), {
                expiration: new Date(Date.now() + 7 * 24 * 3600 * 1000),
            });

            return jsonNoCache(c, { earned: actualEarned, balance: newBalance, duplicate: false });
        } catch (error) {
            console.error('[api/diamond/settle-run] error:', error);
            return jsonError(c, 500, 'Internal server error');
        }
    });

    api.post('/diamond/buy-item', async c => {
        try {
            const username = await reddit.getCurrentUsername();
            if (!username) return jsonError(c, 401, 'Not logged in');

            let body: unknown;
            try {
                body = await c.req.json();
            } catch {
                return jsonError(c, 400, 'Invalid JSON');
            }

            const payload = body as { itemId?: string };
            const itemId = payload.itemId;
            if (!itemId || typeof itemId !== 'string') {
                return jsonError(c, 400, 'Invalid itemId');
            }

            const price = DIAMOND_ITEM_PRICE;
            const balanceKey = `${DIAMOND_BALANCE_PREFIX}${username}`;
            const currentBalance = await ensureDiamondBalance(username);

            if (currentBalance < price) {
                return jsonError(c, 400, 'Insufficient diamonds');
            }

            const newBalance = currentBalance - price;
            await redis.set(balanceKey, String(newBalance));

            return jsonNoCache(c, { success: true, itemId, price, balance: newBalance });
        } catch (error) {
            console.error('[api/diamond/buy-item] error:', error);
            return jsonError(c, 500, 'Internal server error');
        }
    });

    api.post('/subscribe', async c => {
        try {
```

```
            const username = await reddit.getCurrentUsername();
            let alreadySubscribed = false;
            let diamondsGranted = 0;
            let newBalance = 0;
            if (username) {
                const existing = await redis.hGet(FOLLOW_KEY, username);
                alreadySubscribed = existing === '1';
                if (!alreadySubscribed) {
                    await redis.hSet(FOLLOW_KEY, { [username]: '1' });
                    // Grant subscribe reward diamonds
                    const balanceKey = `${DIAMOND_BALANCE_PREFIX}${username}`;
                    const currentBalance = await ensureDiamondBalance(username);
                    newBalance = currentBalance + SUBSCRIBE_DIAMOND_REWARD;
                    await redis.set(balanceKey, String(newBalance));
                    diamondsGranted = SUBSCRIBE_DIAMOND_REWARD;
                } else {
                    newBalance = await ensureDiamondBalance(username);
                }
            }
            return jsonNoCache(c, { success: true, alreadySubscribed, diamondsGranted, newBalance });
        } catch (error) {
            console.error('[api/subscribe] error:', error);
            return jsonError(c, 500, 'Internal server error');
        }
    });

    // ===== FILE: devvit/src/server/routes/menu.ts =====
    import { Hono } from 'hono';
    import { context, reddit } from '@devvit/web/server';

    export const menu = new Hono();

    menu.post('/create-post', async c => {
        try {
            const post = await reddit.submitCustomPost({
                title: '🤖 Granny vs Robot — How Far Can You Go?',
            });

            return c.json(
                {
                    navigateTo: `https://reddit.com/r/${context.subredditName}/comments/${post.id}`,
                },
                200
            );
        } catch (error) {
            console.error('[menu/create-post] error:', error);
            return c.json({ showToast: 'Failed to create post' }, 400);
        }
    });

```